

“ResuMind: An AI Powered Resume Analyzer”

**A Major Project Report Submitted to
Rajiv Gandhi Proudhyogiki Vishwavidyalaya**



**Towards Partial Fulfillment for the Award of
Bachelor of Engineering in Computer Science Engineering**

Submitted by:

**Devansh Pipraiya (0827CS221083)
Hardeep Kumar (0827CS221099)
Harsh Verma (0827CS221103)**

Guided by:

**Prof. Kavita Namdev
Computer Science and Engineering**



***Acropolis Institute of Technology & Research, Indore
Jan-June 2026***

EXAMINER APPROVAL

The Major Project entitled “**ResuMind: An AI Powered Resume Analyzer**” *submitted* by **Devansh Pipraiya (0827CS221083)**, **Hardeep Kumar (0827CS221099)**, **Harsh Verma (0827CS221103)** has been examined and is hereby approved towards partial fulfillment for the award of *Bachelor of Technology degree in Computer Science Engineering* discipline, for which it has been submitted. It is understood that by this approval the undersigned does not necessarily endorse or approve any statement made, opinion expressed, or conclusion drawn therein, but approve the project only for the purpose for which it has been submitted.

(Internal Examiner)

Date:

(External Examiner)

Date:

RECOMMENDATION

This is to certify that the work embodied in this major project entitled “**ResuMind: An AI Powered Resume Analyzer**” was submitted by **Devansh Pipraiya (0827CS221083), Hardeep Kumar (0827CS221099), Harsh Verma (0827CS221103)** is a satisfactory account of the bonafide work done under the supervision of **Prof. Kavita Namdev, Assistant Professor, CSE, Acropolis Institute of Technology and Research** is recommended towards partial fulfillment for the award of the Bachelor of Technology (Computer Science Engineering) degree by Rajiv Gandhi Proudyogiki Vishwavidhyalaya, Bhopal.

Prof. Kavita Namdev
(Project Coordinator)

STUDENTS UNDERTAKING

This is to certify that the major project entitled **“ResuMind: An AI Powered Resume Analyzer”** has been developed by us under the supervision of **Prof. Kavita Namdev, Assistant Professor, CSE, Acropolis Institute of Technology and Research**. The whole responsibility for the work done on this project is ours. The sole intension of this work is only for practical learning and research.

We further declare that to the best of our knowledge; this report does not contain any part of any work which has been submitted for the award of any degree either in this University or in any other University / Deemed University without proper citation and if the same work is found then we are liable for explanation to this.

Devansh Pipraiya (0827CS221083)

Hardeep Kumar (0827CS221099)

Harsh Verma (0827CS221103)

Acknowledgement

We thank the almighty Lord for giving me the strength and courage to sail out through the tough and reach shore safely. There are a number of people without whom this project would not have been feasible. Their high academic standards and personal integrity provided me with continuous guidance and support.

We owe a debt of sincere gratitude, deep sense of reverence and respect to our guide and mentor **Prof. Kavita Namdev, Assistant Professor, CSE, Acropolis Institute of Technology and Research, Indore** for his motivation, sagacious guidance, constant encouragement, vigilant supervision, and valuable critical appreciation throughout this project work.

We express profound gratitude and heartfelt thanks to **Dr Kamal Kumar Sethi, Professor & Head, CSE, Acropolis Institute of Technology and Research, Indore** for his support, suggestion, and inspiration for carrying out this project. I am very thankful to other faculty and staff members of the department for providing me with all the support, help and advice during the project. We would be failing in our duty if we did not acknowledge the support and guidance received from **Dr Shashi Chandra Sharma, Director, CSE, Acropolis Institute of Technology and Research, Indore** whenever needed. We take the opportunity to convey my regards to the management of Acropolis Institute, Indore, for extending academic and administrative support and providing me all necessary facilities for the project to achieve our objectives.

We are grateful to **our parents and family members** who have always loved and supported us unconditionally. To all of them, we want to say, “Thank you”, for being the best family that one could ever have and without whom none of this would have been possible

Devansh Pipraiya (0827CS221083)

Hardeep Kumar (0827CS221099)

Harsh Verma (0827CS221103)

Executive Summary

ResuMind: An AI Powered Resume Analyzer

This project is submitted to Rajiv Gandhi Proudhyogiki Vishwavidhyalaya, Bhopal (MP), India for partial fulfillment of the Bachelor of Technology in Computer Science Engineering branch under the sagacious guidance and vigilant supervision of ***Prof. Kavita Namdev, Assistant Professor, CSE, AITR.***

The project is based on AI-powered resume analysis. It is a web application developed using modern frontend technologies including React 19, React Router v7, TypeScript, Tailwind CSS, and the Puter.js platform. The purpose of this project is to solve the major problem faced by job seekers whose resumes are silently rejected by Applicant Tracking Systems (ATS) before any human review. ResuMind allows users to upload their PDF resumes, input a job description, and instantly receive an ATS compatibility score, matched skills, missing keywords, and AI-generated improvement suggestions — all without any backend server or database.

Keywords: Resume Analysis, Applicant Tracking System (ATS), AI Feedback, Puter.js, React, Serverless Architecture, Keyword Matching, PDF Parsing, Job Matching, TypeScript.

List of Figures

Figure 3-1: Block Diagram	11
Figure 3-2: Use Case Diagram	13
Figure 3-3: Activity Diagram	14
Figure 3-4: Data Flow Diagram Level 0.....	15
Figure 3-5: Data Flow Diagram Level 1	15
Figure 3-6: Data Flow Diagram Level 2.....	16
Figure 4-1: ResuMind Dashboard	20
Figure 4-2: ResuMind Resume Upload Dashboard.....	20
Figure 4-3: ATS Score Screenshot	21
Figure 4-4: Categorized Feedback Screenshot	22

Table of Contents

CHAPTER 1. INTRODUCTION	1
1.1 Overview	1
1.2 Background and Motivation	2
1.3 Problem Statement and Objectives	3
1.4 Scope of Project	3
1.5 Team Organization	4
1.6 Report Structure.....	4
CHAPTER 2. REVIEW OF LITERATURE	6
2.1 Preliminary Investigation	6
2.1.1 Current System	6
2.2 Limitations of Current System.....	7
2.3 Requirement Identification and Analysis for Project	7
2.4 Conclusion.....	8
CHAPTER 3. PROPOSED SYSTEM	9
3.1 The Proposal.....	9
3.2 Benefits of the Proposed System	9
3.3 Block Diagram.....	10
3.4 Feasibility Study	11
3.4.1 Technical	11
3.4.2 Economical	12
3.4.3 Operational	12
3.5 Design Representation	12
3.5.1 Module	12
3.5.2 UML Diagrams	13
3.6 Deployment Requirements.....	16
3.6.1 Hardware	16
3.6.2 Software	16

CHAPTER 4. IMPLEMENTATION	17
4.1 Technique Used	17
4.2 Tools and Languages used	17
4.2.1 React	19
4.2.2 React Router v7	18
4.2.3 TypeScript	18
4.2.4 Tailwind CSS	18
4.2.5 Puter.js	18
4.2.6 Vite	19
4.2.7 Client-Side PDF Processing	19
4.2.8 Git and GitHub	19
4.3 User Interface	19
4.4 Testing	22
CHAPTER 5. CONCLUSION	24
5.1 Conclusion	24
5.2 Limitations of the work	24
5.3 Suggestion and Recommendations for Future Work	25
BIBLIOGRAPHY	26
SOURCE CODE	27

Chapter 1

Introduction

The recruitment Process has undergone a dramatic transformation in recent years. Organizations across industries now rely on Applicant Tracking Systems (ATS) to manage the overwhelming volume of job applications they receive. These automated systems filter and rank resumes based on keyword matching, formatting rules, and alignment with job descriptions — often before any human recruiter ever reviews a single application. While this process benefits employers by reducing manual screening effort, it creates a significant and largely invisible barrier for job seekers.

Candidates invest considerable time crafting their resumes, yet receive no feedback when their applications are rejected. They are left unaware of whether their resume failed due to missing keywords, poor formatting, lack of relevant skills, or simple structural issues that an ATS cannot parse. This opacity results in a cycle of repeated rejections that candidates cannot diagnose or fix.

ResuMind is developed to break this cycle. It is a fully browser-based, AI-powered resume analysis platform that empowers job seekers to understand exactly how their resume performs against any specific job description. By leveraging modern frontend technologies and the Puter.js platform, ResuMind delivers instant ATS compatibility scoring, keyword gap analysis, and actionable AI-generated improvement suggestions — all without requiring any backend server, database, or technical setup from the user.

1.1 Overview

The modern job market is increasingly competitive, and candidates often struggle to understand why their resumes fail to pass Applicant Tracking

Systems (ATS) before a human recruiter ever reads them. Industry research suggests that up to 75% of resumes are automatically filtered out by ATS platforms based on keyword mismatches, formatting issues, and poor alignment with job descriptions.

ResuMind has been developed to address this problem directly. It is a fully browser-based, AI-powered resume analysis platform that allows users to upload their PDF resumes, paste a job description, and instantly receive structured feedback. The system evaluates how well a candidate's profile aligns with the target role and provides an ATS compatibility score along with matched skills, missing keywords, and AI-generated improvement suggestions — all without requiring any backend server, database, or API key management.

1.2 Background and Motivation

The rise of Applicant Tracking Systems in modern recruitment has created a hidden barrier for job seekers. ATS platforms filter resumes based on keyword matching algorithms, rejecting candidates who may be qualified but whose resumes lack the specific terminology used in a job description. Most ATS tools provide no feedback to the candidate, leaving them unable to understand or correct the reasons for rejection.

Existing commercial tools such as Resumeworded, Jobscan, and Kickresume partially address this problem, but they either require paid subscriptions, lack deep AI-driven contextual feedback, or do not offer centralized resume storage. There is no freely accessible, job-specific, AI-powered resume analysis tool that runs entirely in the browser without backend infrastructure.

ResuMind was motivated by this gap. By combining the capabilities of large language models with a fully client-side architecture powered by Puter.js, it is possible to deliver accurate, personalized resume feedback at zero cost and with minimal setup for the end user.

1.3 Problem Statement and Objectives

Problem Statement: Job seekers frequently submit applications without understanding why their resumes are rejected. ATS filtering is opaque, generic resume advice does not account for specific job descriptions, and no centralized browser-based storage solution exists for managing multiple resume versions without a backend.

1. **Objective 1:** To develop a browser-based platform that allows users to upload PDF resumes and analyze them against specific job descriptions using AI, providing an ATS compatibility score.
2. **Objective 2:** To identify and display matched skills and missing keywords by comparing the resume content to the requirements stated in the job description.
3. **Objective 3:** To provide AI-generated, section-specific improvement suggestions in plain language that are tailored to the target role.
4. **Objective 4:** To implement secure, user-scoped resume storage using Puter.js cloud file storage so that users can manage multiple resume versions without a backend.
5. **Objective 4:** To enable an iterative improvement workflow by persisting past analysis results so users can re-analyze resumes against different job descriptions and track progress over time.

1.4 Scope of the Project

This project covers the development of a fully client-side web application built with React 19 and React Router v7. The scope includes:

Users can authenticate entirely through Puter.js without any backend. They can upload PDF resumes which are stored in Puter's cloud file system. They can submit a job description alongside a selected resume and receive an AI-generated analysis that includes an ATS score, matched skills, missing keywords, and improvement suggestions. The application stores past analysis

results so users can review previous feedback at any time. The interface is fully responsive and works across desktop and mobile browsers.

The project does not cover building a custom backend server, training a custom machine learning model, or integrating with actual job portals. The AI analysis is powered by Puter.js which internally routes requests to large language models. Resume parsing is handled client-side through PDF text extraction.

1.5 Team Organization

- **Devansh Pipraiya:**

Led the frontend development of the application using React 19 and React Router v7. Responsible for the overall system architecture, Puter.js integration, AI prompt engineering, and PDF processing layer. Also carried out preliminary investigation and requirement analysis.

- **Hardeep Kumar:**

Responsible for data collection, testing, and quality assurance. Conducted unit testing, integration testing, and user acceptance testing of the platform. Also contributed to preparing the final project report and presentation materials.

- **Harsh Verma:**

Conducted in-depth research on existing resume analysis tools and ATS systems, analyzing their advantages and disadvantages to inform the system design. Also contributed to frontend development, UI component implementation, and project documentation.

1.6 Report Structure

The project **ResuMind** is primarily concerned with AI-powered resume analysis, and the project report is categorized into five chapters..

Chapter 1: Introduction — introduces the background and motivation, problem statement, objectives, scope, team organization, and report outline.

Chapter 2: Review of Literature — surveys existing resume analysis systems, discusses their limitations, and identifies the requirements for the proposed system.

Chapter 3: Proposed System — presents the system proposal, benefits, block diagram, feasibility study, design representations including UML diagrams, and deployment requirements.

Chapter 4: Implementation — details the technology stack, tools, programming languages, user interface design, and testing strategy.

Chapter 5: Conclusion — summarizes the results, limitations of the current work, and recommendations for future enhancements.

Chapter 2

Review of Literature

The increasing adoption of Applicant Tracking Systems by employers has created a significant challenge for job seekers. ATS platforms screen resumes automatically before any human review takes place, filtering candidates based on keyword presence, formatting compliance, and alignment with job descriptions. Despite the widespread use of these systems, most candidates receive no feedback explaining why their resume was rejected.

A review of existing resume analysis and career tools reveals that while several platforms attempt to address this problem, none provides a complete solution that is free, browser-based, job-specific, and capable of storing resumes without a backend.

2.1 Preliminary Investigation

2.1.1 Current System

Several existing systems have been studied:

- **Resumeworded:** Provides ATS scoring and line-by-line resume feedback. However, it limits free usage, requires account creation on its own platform, and does not offer in-browser resume storage across sessions. Feedback is not deeply tailored to the specific job description provided.
- **JobScan:** Offers side-by-side keyword comparison between a resume and a job description. Requires a paid subscription for full features. Provides no AI-generated written feedback — analysis is limited to keyword matching with no contextual suggestions.
- **KickResume:** Focuses on resume building and template-based creation rather than job-specific analysis. Cannot analyze an existing resume against a

specific job description. Provides no ATS score, no client-side storage, and no AI-driven feedback.

- **Zety Resume Checker:** Reviews resumes for formatting and structure. Analysis is generic and not tied to any specific job description. Offers no ATS simulation, no keyword gap analysis, and no AI-generated improvement suggestions.

2.2 Limitations of Current System

The limitations identified across existing tools are as follows:

- Existing tools do not provide deep, job-specific AI feedback tailored to the exact requirements of an individual job posting.
- Most tools require account creation on proprietary platforms and do not offer in-browser resume storage without backend infrastructure.
- Full-featured analysis is locked behind paywalls, making the tools inaccessible to many job seekers.
- No existing tool provides a unified platform for storing, reviewing, and re-analyzing multiple resume versions in a single browser-based session.
- None of the reviewed tools operates in a fully serverless, client-side manner — all require backend infrastructure for at least some functionality.

2.3 Requirement Identification and Analysis for Project

Based on the review of literature and the identified limitations of existing systems, the following requirements were derived for ResuMind:

2.3.1 Functional Requirements:

- Allow users to authenticate and access the platform without any custom backend.
- Allow users to upload PDF resumes and store them securely in cloud storage.
- Accept a job description as input and extract text from the uploaded PDF on the client side.
- Send a structured AI prompt and receive a JSON response containing ATS

score, matched skills, missing keywords, and improvement suggestions.

- Display analysis results in a clean, categorized UI.
- Persist past analysis results so users can revisit them without re-running the AI.

2.3.2 Non-Functional Requirements:

- **Performance:** The application must return AI analysis results within an acceptable time limit (target: under 15 seconds) under normal network conditions.
- **Security:** Resume files and analysis results must remain private to the authenticated user. No data must be accessible across sessions or user accounts.
- **Usability:** The interface must be intuitive and accessible to users without technical knowledge, working across desktop and mobile devices.
- **Reliability:** The system must handle edge cases such as scanned PDFs, empty job descriptions, and malformed AI responses gracefully.

2.4 Conclusion

This chapter reviewed the existing resume analysis tools and identified key gaps that motivated the development of ResuMind. The review confirms that no currently available free tool provides job-specific AI feedback, client-side resume storage, and a fully serverless architecture simultaneously. These findings directly informed the design decisions of the proposed system.

Chapter 3

Proposed System

3.1 The Proposal

To address the limitations of existing systems, ResuMind proposes a fully client-side, AI-powered resume analysis platform. The system allows users to authenticate via Puter.js, upload and store PDF resumes in cloud storage, and analyze them against any job description using a large language model. The AI returns a structured ATS score, matched skills, missing keywords, and section-specific improvement suggestions — all without any backend server, database, or API key exposure.

3.2 Benefits of the Proposed System

The current system had a lot of challenges that are overcome by this system:

- **Free and Accessible:** ResuMind requires no subscription or payment. Any user with a browser and internet connection can use it.
- **Job-Specific Feedback:** Unlike generic resume checkers, ResuMind analyzes a resume against the specific job description provided by the user, making the feedback directly actionable.
- **No Backend Required:** By using Puter.js for authentication, file storage, and AI API access, the entire application runs client-side, eliminating infrastructure complexity and cost.
- **Data Privacy:** Resume files are stored under the user's own Puter account and are never shared with other users or third-party servers beyond the Puter.js platform.
- **Iterative Improvement:** Past analysis results are stored and accessible at any time, enabling users to track improvements across multiple resume versions and job applications.

3.3 Block Diagram / System Architecture

ResuMind follows a layered, client-side architecture with no dedicated backend server. All data flows through Puter.js APIs running in the browser.

The system is organized into four logical layers:

- **Frontend Layer:** Built with React 19, React Router v7, Tailwind CSS, and TypeScript. Handles all user interactions, routing, and UI rendering.
- **Puter.js Platform Layer:** Provides browser-based authentication, cloud file storage, key-value data persistence, and AI API access. Replaces all traditional backend services.
- **AI Processing Layer:** When analysis is triggered, the client extracts PDF text, constructs a structured prompt, and sends it to an LLM via the Puter.js AI API. The JSON response is parsed and validated before display.
- **Client-Side PDF Layer:** PDF text extraction is performed entirely in the browser before any AI call. Handles edge cases for scanned or image-based PDFs.

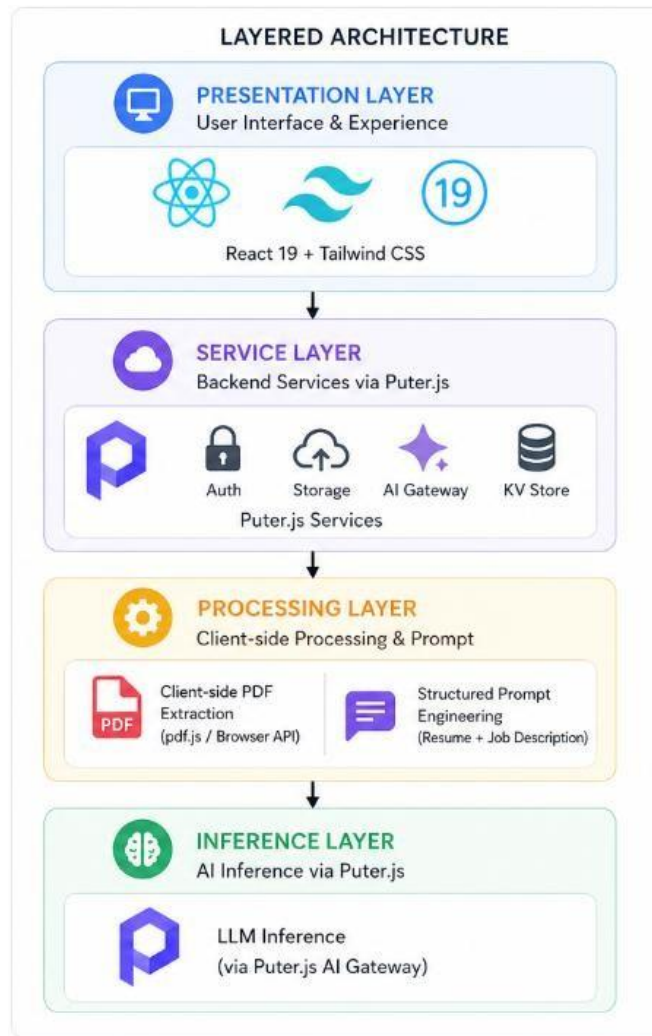


Figure 3-1 : System Architecture Diagram

3.4 Feasibility Study

A feasibility study is an analysis of how successfully a system can be implemented, [6] accounting for factors that affect it such as economic, technical, and operational factors to determine its potential positive and negative outcomes before investing a considerable amount of time and money into it.

3.4.1 Technical

The project uses well-established technologies — React 19, TypeScript, Tailwind CSS, and Puter.js — all of which are production-ready and widely supported. Browser-based PDF text extraction and client-side AI API calls are

both proven approaches. The serverless architecture eliminates infrastructure provisioning challenges.

3.4.2 Economical

Since ResuMind is fully serverless and uses Puter.js for AI routing and storage, there are no server hosting costs, no database licensing fees, and no AI API key costs for the developer. The application is freely accessible to all users without a subscription.

3.4.3 Operational

The system is designed to be intuitive for job seekers with no technical background. A single-page dashboard, straightforward upload flow, and clearly structured results page ensure that all users can operate the platform without guidance.

3.5 Design Representation

3.5.1 Modules: -

- **Authentication Module:** Handles user login via the Puter.js built-in authentication system. Checks for an active session on app load and prompts the user to sign in if no session exists. No passwords or tokens are managed by the application.
- **Resume Upload and Storage Module:** Allows users to upload PDF resumes via a file picker. Uploaded files are stored in Puter's cloud file system. Metadata including filename, upload date, and analysis result keys are stored in Puter's key-value store (KV).
- **Dashboard Module:** Displays all uploaded resumes as cards with metadata. Provides quick-access actions to view, re-analyze, or delete each resume. Acts as the central hub of the user experience.
- **AI Analysis Module:** Extracts raw text from the selected PDF on the client side. Constructs a structured prompt combining the resume text and job description. Sends the prompt to the Puter.js AI API and parses the JSON

response containing ATS score, matched skills, missing keywords, experience alignment, and improvement suggestions.

- **Results Display Module:** Renders the ATS score with a color-coded visual indicator (green/amber/red). Displays matched skills, missing keywords, and improvement suggestions in clearly separated sections.
- **Resume History Module:** Retrieves and displays past analysis results stored in Puter KV. Allows users to re-run analysis against new job descriptions without re-uploading the file.

3.5.2 UML Diagrams:

- **Use Case Diagram:**

The system has one primary actor — the User. Use cases include: Sign In via Puter, Upload Resume, View Dashboard, Select Resume, Enter Job Description, Trigger Analysis, View ATS Score, View Matched Skills, View Missing Keywords, View Improvement Suggestions, View Past Analysis Results, and Delete Resume.

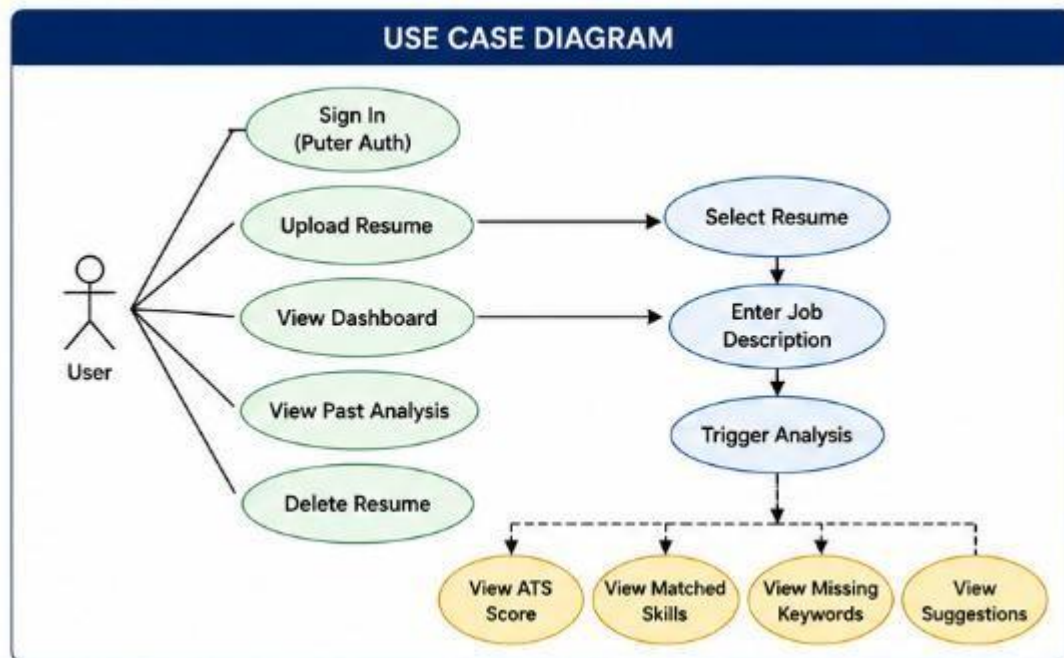


Figure 3-2: Use Case Diagram

- **Activity Diagram:**

This activity flow begins with the user visiting the application and authenticating via Puter.js. The user is directed to the Dashboard, where existing resumes are listed. The user uploads a new resume or selects an existing one, pastes a job description, and triggers the analysis. The system extracts PDF text, constructs the AI prompt, calls the Puter.js AI API, receives the JSON response, stores the result in KV, and renders the results page.

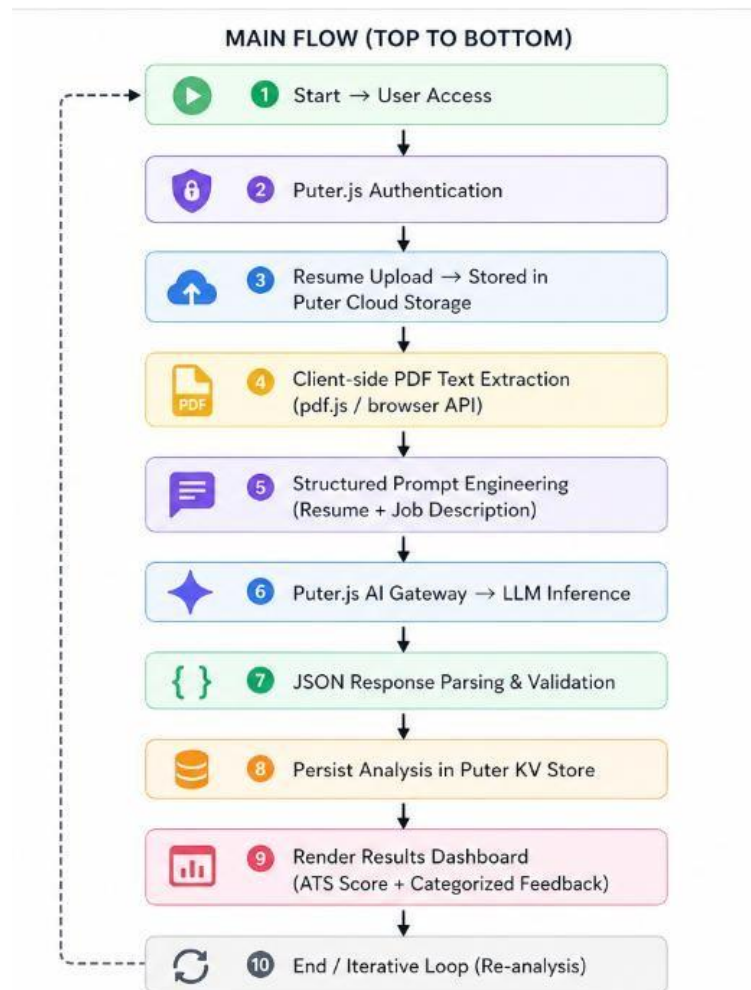


Figure 3-3: Activity Diagram

- **Data Flow Diagram:**

The user provides a resume (PDF) and job description as inputs. The system outputs an ATS score, matched skills, missing keywords, and improvement suggestions.

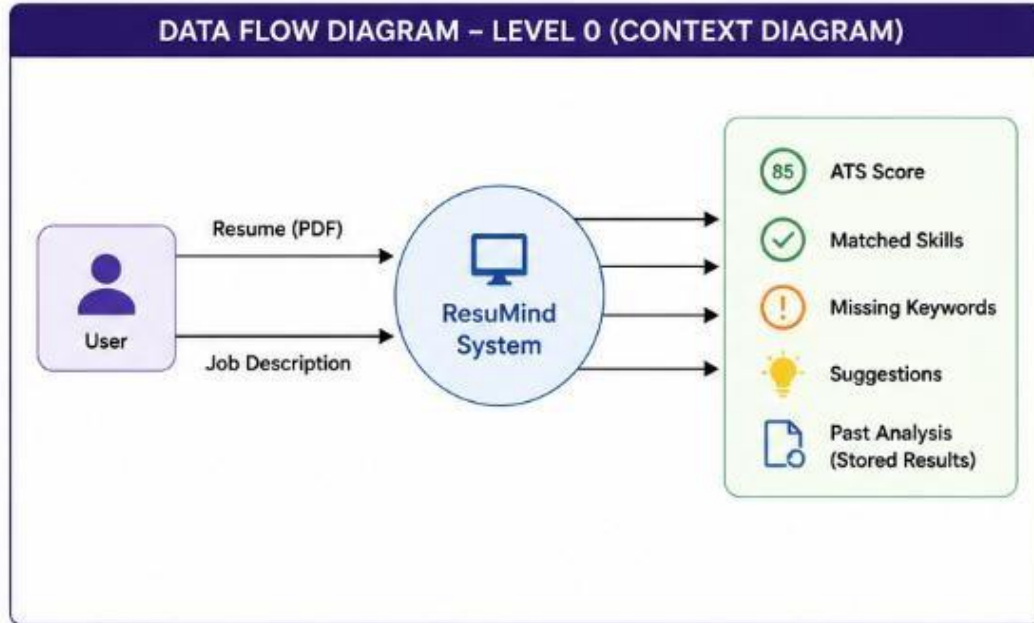


Figure 3-4: Data Flow Diagram Level-0

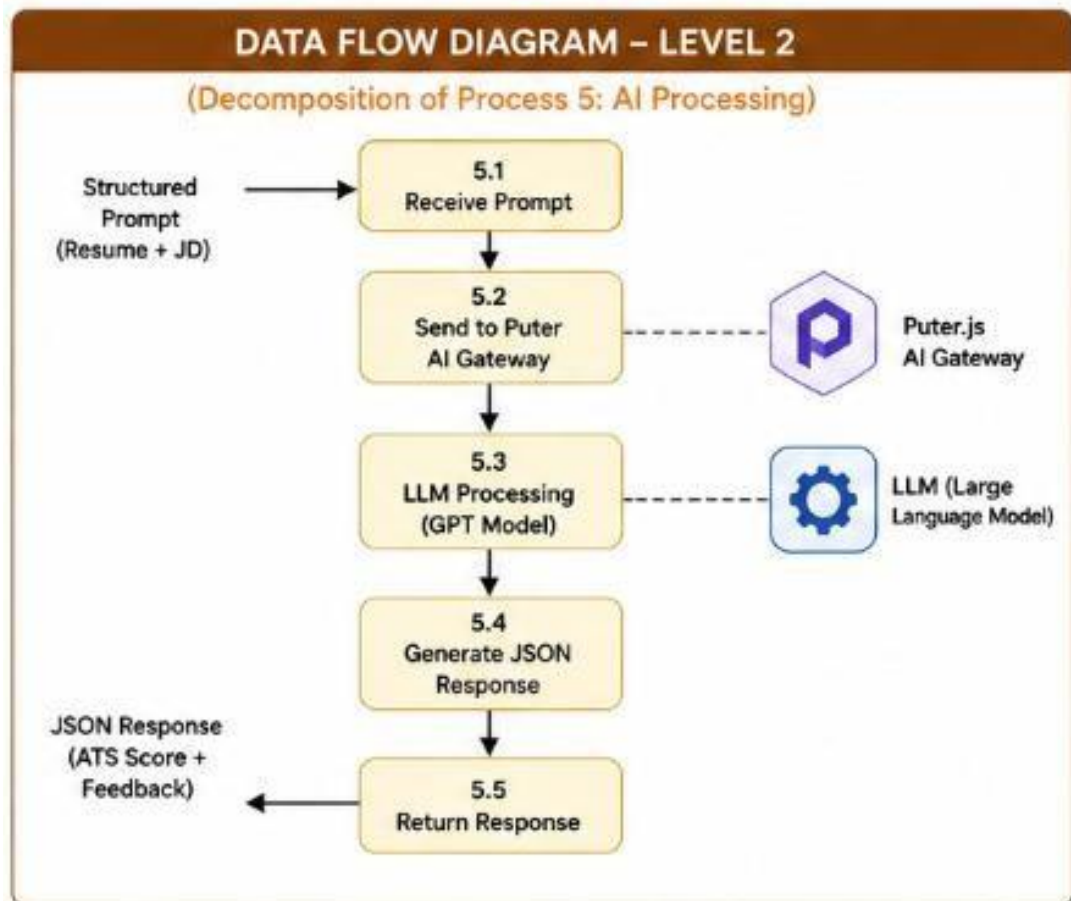


Figure 3-5: Data Flow Diagram Level-1

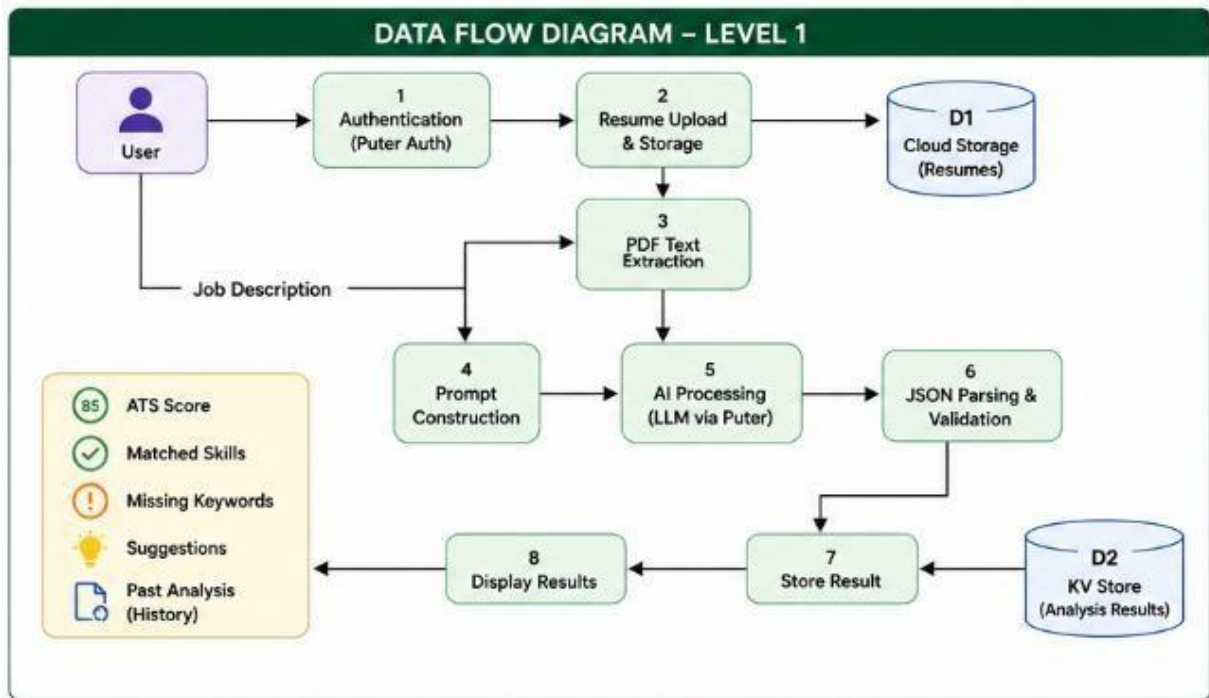


Figure 3-6: Data Flow Diagram Level-2

3.6 Deployment Requirements

There are various requirements (hardware, software, and services) to successfully deploy the system. These are mentioned below:

3.6.1 Hardware

- Any modern desktop, laptop, or mobile device with a web
- Minimum 4 GB RAM recommended.
- Stable internet connection (for Puter.js API calls and AI processing)

3.6.2 Software

- Modern web browser (Chrome, Firefox, Edge, Safari — latest versions)
- Node.js v18 or above (for local development only)
- npm (for package management in development)
- Vite (build tool, included as a dev dependency)
- React 19, React Router v7, Tailwind CSS, TypeScript (all included as project dependencies)
- Puter.js SDK (loaded via CDN or npm)

Chapter 4

Implementation

4.1 Technique Used

The ResuMind implements AI-powered resume analysis using a fully client-side, serverless architecture. The core technique involves structured prompt engineering to extract consistent, JSON-formatted analysis from a large language model.

When the user triggers an analysis, the application first extracts raw text from the uploaded PDF directly in the browser using a client-side PDF parsing library. This extracted text is combined with the user-provided job description and formatted into a carefully engineered prompt that instructs the AI model to evaluate the resume against the job description and return a strict JSON object. The prompt specifies the exact output schema — including an ATS score out of 100, matched skills, missing keywords, experience alignment, and improvement suggestions — ensuring consistent, parseable responses across all analyses.

The entire data flow from PDF upload to AI response and result display happens without any server-side processing. Puter.js routes AI requests to large language models internally, handles user authentication, and manages cloud file storage, making the application fully serverless while still delivering production-grade functionality.

4.2 Tools and Languages Used:

4.2.1 React 19

React is the primary UI library used to build ResuMind's interface. React 19 provides component-based architecture, enabling reusable UI elements for the

dashboard, resume detail view, and results page. React's declarative model simplifies managing the application state across the upload, analysis, and result display flows.

4.2.2 React Router v7

React Router v7 is used in framework mode, which enables server-side-rendered-style routing with loaders and actions entirely on the client. Route loaders fetch resume data and analysis results before rendering each page, ensuring that the UI is always populated with the correct data before it appears to the user.

4.2.3 TypeScript

TypeScript is used throughout the project for static type checking. All components, utility functions, route loaders, and API response parsers are fully typed, reducing runtime errors and improving code maintainability. Type definitions for the AI response schema ensure that the results object is always handled correctly across the codebase.

4.2.4 Tailwind CSS

Tailwind CSS is the utility-first CSS framework used for all styling in ResuMind. It enables rapid, consistent, and responsive UI development without writing custom CSS files. Tailwind's design tokens ensure visual consistency across all components and screen sizes.

4.2.5 Puter.js

Puter.js is a client-side JavaScript SDK that provides authentication, cloud file storage, key-value data persistence, and AI API access directly in the browser. It replaces all traditional backend services — no Express server, no database, and no API key management are required. Puter.js handles AI model routing internally, meaning the developer does not expose any API keys in the frontend code.

4.2.6 Vite

Vite is the build tool and development server used for the project. It provides extremely fast hot module replacement (HMR) during development and optimized production builds with minimal configuration. The vite.config.ts file in the project root configures the build pipeline.

4.2.7 Client-Side PDF Processing

PDF text extraction is performed directly in the browser using a client-side PDF parsing approach before any AI call is made. The extracted text is cleaned and truncated to fit within the AI model's context window. For scanned or image-based PDFs where text extraction returns empty results, the user is notified and prompted to upload a text-based PDF resume.

4.2.8 Git and GitHub

Version control is managed using Git, with the project hosted on GitHub at:

https://github.com/Devansh-pipraiya/ResuMind_Major_Project

Development Environment

- **Visual Studio Code:** Primary development IDE.
- **npm:** Package management and script execution.

4.3 User Interface

- **Resume Detail View**

After selecting a resume, the user is taken to the Resume Detail view. This two-column layout shows the extracted text from the resume on one side and a job description input area on the other. An "Analyze" button triggers the AI analysis flow.

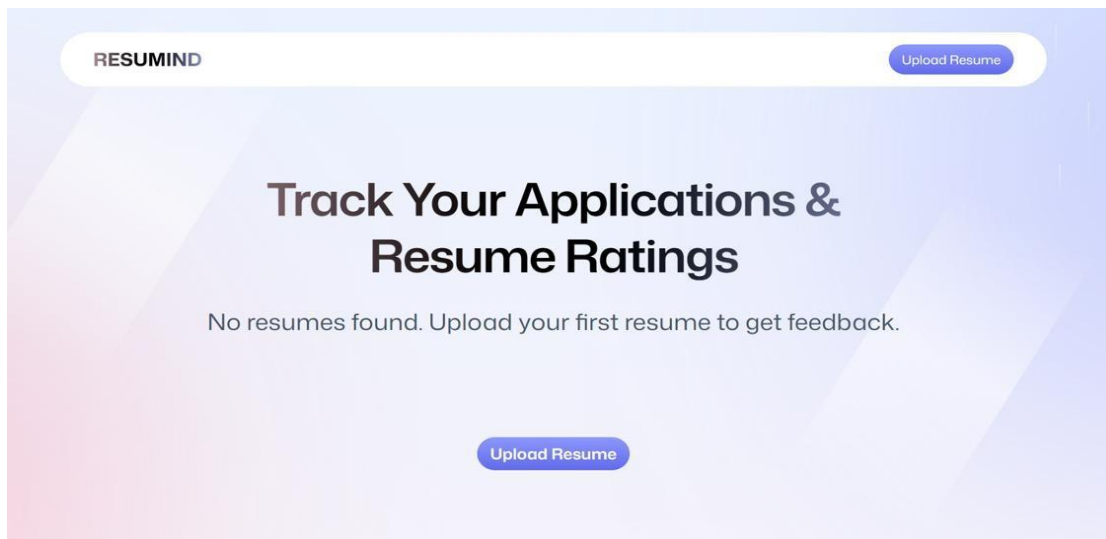


Figure 4-1: ResuMind Dashboard

The image shows the 'Resume Upload Dashboard' within the ResuMind application. It features a similar header to the main dashboard, with the 'RESUMIND' logo and an 'Upload Resume' button. The main heading is 'Smart feedback for your dream job'. Below this, a sub-heading says 'Drop your resume for an ATS score and improvement tips'. The form consists of several input fields: 'Company Name' (with a placeholder 'Company Name'), 'Job Title' (with a placeholder 'Job Title'), and 'Job Description' (a larger text area with a placeholder 'Job Description'). Below these fields is an 'Upload Resume' section, which includes a large light blue box with a circular icon containing an 'i' and the text 'Click to upload or drag and drop PDF (max 20 MB)'. At the bottom of the form, there is a prominent blue button labeled 'Analyze Resume'.

Figure 4-2: ResuMind Resume Upload Dashboard

- **Analysis Result View**

The Analysis Result view displays:

- **ATS Score Card:** A prominent numerical score out of 100 with a color-coded indicator (green for high alignment ≥ 70 , amber for moderate 40–69, red for low < 40).
- **Matched Skills Section:** Lists keywords and competencies present in both the resume and the job description, showing the candidate's strengths.
- **Missing Keywords Section:** Highlights terms present in the job description but absent from the resume, guiding the user on what to add.
- **Improvement Suggestions:** AI-generated, section-specific guidance written in plain language, such as recommending specific tools to mention or how to rephrase experience bullet points.

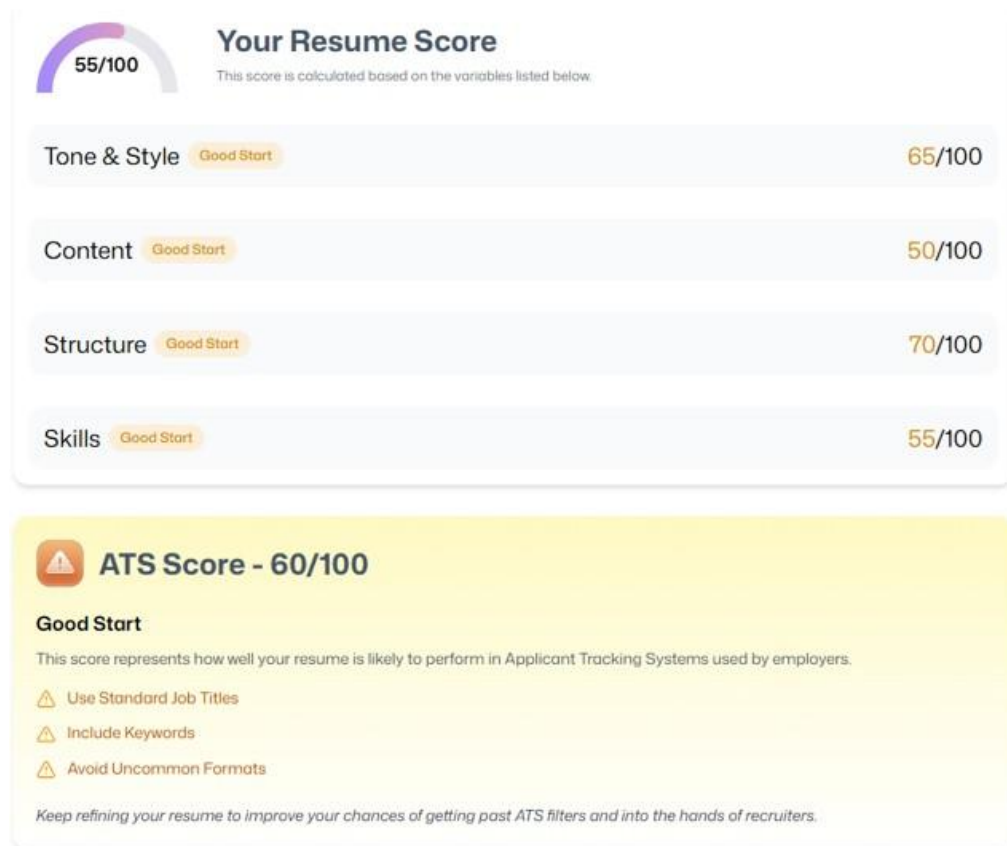


Figure 4-3: ATS Score Screenshot

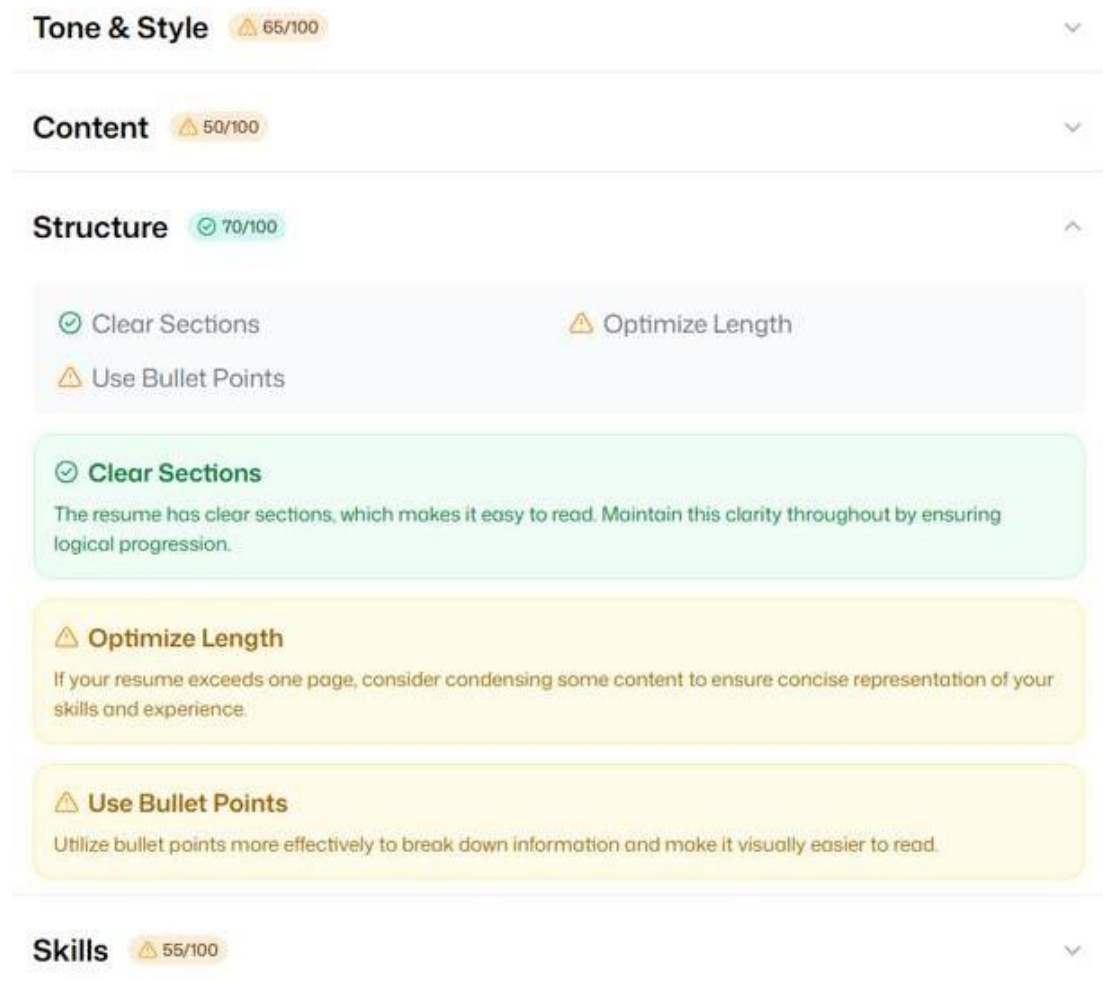


Figure 4-4: Categorized Feedback Screenshot

4.4 Testing

- **Unit Testing**

Individual components and utility functions were tested in isolation. Tests covered PDF text extraction accuracy, prompt construction correctness, JSON response parsing and validation, and React component rendering. Each module was verified to function correctly before integration into the complete application flow.

- **Integration Testing**

End-to-end integration tests validated that Puter.js authentication, file storage, key-value persistence, and AI API calls function correctly together. A simulated user session — sign in, upload resume, trigger analysis, retrieve results — was

run repeatedly to confirm that data flows without errors across all system boundaries.

- **Functional Testing**

All user-facing flows were manually tested including login, resume upload, job description input, analysis triggering, and result rendering. Edge cases including malformed PDFs, empty job descriptions, and unexpected AI response formats were handled with appropriate error messages.

- **User Acceptance Testing**

Test users uploaded their actual resumes against real job postings from major job boards. Feedback was collected on the relevance of matched skills, the accuracy of missing keyword identification, and the usefulness of AI improvement suggestions. Users reported that the ATS score and sectioned feedback were actionable and significantly clearer than generic resume advice.

- **Performance Testing**

Performance testing confirmed that the application processes PDF files up to 5MB in size and returns AI results within acceptable time limits (5–15 seconds under normal network conditions). The application remained responsive during back-to-back analysis requests.

- **Security Testing**

Security testing verified that resume files and analysis results remain private to the authenticated Puter user. No data was found to be accessible across different user sessions. The absence of API key exposure in the client-side code was confirmed.

- **Regression Testing**

All functional and integration tests were re-run after each significant code change to ensure that new additions did not break existing functionality. The application remained stable throughout the development cycle.

Chapter 5

Conclusion

5.1 Conclusion

ResuMind successfully demonstrates that a fully functional, AI-powered resume analysis platform can be built without any backend infrastructure. By combining React 19, React Router v7, TypeScript, Tailwind CSS, and the Puter.js platform, the project delivers a practical, free tool that provides job seekers with real, job-specific feedback on their resumes in seconds.

The platform addresses a genuine and widespread problem — the opacity of ATS filtering and the absence of personalized resume feedback — and solves it with a clean, serverless, client-side architecture. All five project objectives have been successfully achieved: users can authenticate, upload resumes, analyze them against job descriptions, receive structured AI feedback, and revisit past analyses through a persistent resume history.

The use of Puter.js as the platform layer removes the usual complexity of authentication, file storage, and AI integration, resulting in a codebase that is lean, maintainable, and easily extensible. The project establishes that modern browser capabilities and client-side AI APIs are sufficient for complex document analysis tasks that previously required significant backend infrastructure.

5.2 Limitations of the Work

- The quality and accuracy of the AI-generated analysis depends on the underlying language model routed by Puter.js, which the application does not control directly.
- Client-side PDF text extraction does not support scanned or image-based PDFs. Users with image-only resume PDFs must convert their files before

use.

- The AI context window limits the length of resumes and job descriptions that can be analyzed simultaneously. Very long documents may be truncated before analysis.
- The application currently supports PDF format only. Other resume formats such as .docx are not directly supported.
- The ATS score is an AI-estimated approximation and does not reflect the behavior of any specific proprietary ATS system used by a particular employer.

5.3 Suggestion and Recommendations for Future Work:

- **Support for additional file formats:**

Extend the upload functionality to support .docx, .txt, and other common resume formats, broadening accessibility for users who do not have PDF versions of their resumes.

- **Resume builder integration:**

Add an in-browser resume builder so that users can implement the AI's improvement suggestions directly within the platform without switching to another tool.

- **Multi-language support:**

Extend the AI prompt and UI to support analysis in languages other than English, making the platform accessible to job seekers in non-English job markets.

- **Job board integration:**

Integrate with publicly available job posting APIs so that users can paste a job URL instead of manually copying the job description.

- **Comparison across versions:**

Allow users to compare ATS scores across multiple versions of the same resume to visually track the improvement made with each edit.

- **Custom scoring weights:**

Allow users to prioritize certain sections (e.g., skills vs. experience) in the ATS scoring, making the feedback more relevant to their specific career stage.

Bibliography

- [1] J. Thompson, "The Hidden Cost of ATS Filtering in Modern Recruitment," *Journal of Human Resources Management*, vol. 15, pp. 34–47, 2023.
- [2] M. Roberts and S. Patel, "Barriers to Job Application Success: ATS and Resume Optimization," *Career Development International*, vol. 28, no. 2, 2022.
- [3] A. Singh, "Comparative Analysis of Online Resume Analysis Tools," *International Journal of Computer Applications*, vol. 185, no. 12, pp. 22–29, 2023.
- [4] H. Zhang et al., "AI-Based Resume Screening: Accuracy and Bias Considerations," *IEEE Transactions on Neural Networks*, vol. 34, no. 6, pp. 2901–2913, 2023.
- [5] R. Nair and P. Mehra, "Serverless Frontend Architectures for Complex Web Applications," *IEEE Software*, vol. 40, no. 3, pp. 58–66, 2023.
- [6] Puter.js Documentation, "Puter JavaScript SDK," [Online]. Available: <https://docs.puter.com>
- [7] React Documentation, "React 19 Release," [Online]. Available: <https://react.dev>
- [8] React Router Documentation, "React Router v7 Framework Mode," [Online]. Available: <https://reactrouter.com/home>
- [9] Vite Documentation, "Getting Started," [Online]. Available: <https://vite.dev/guide>
- [10] TypeScript Documentation, "TypeScript Handbook," [Online]. Available: <https://www.typescriptlang.org/docs>

Source code

The complete source code for the ResuMind project is publicly available on GitHub. The project is organized as a single-repository frontend application built with React 19, React Router v7, TypeScript, and Tailwind CSS, using Puter.js for all platform services.

The repository can be accessed at the following URL:

Github Repository:

https://github.com/Devansh-pipraiya/ResuMind_Major_Project